

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/3193661>

The Image Foresting Transform: Theory, Algorithms, and Applications

Article in IEEE Transactions on Pattern Analysis and Machine Intelligence · February 2004

Impact Factor: 5.78 · DOI: 10.1109/TPAMI.2004.1261076 · Source: IEEE Xplore

CITATIONS

310

READS

69

3 authors, including:



Jorge Stolfi

University of Campinas

121 PUBLICATIONS 4,311 CITATIONS

SEE PROFILE



Roberto de Alencar Lotufo

University of Campinas

184 PUBLICATIONS 3,219 CITATIONS

SEE PROFILE

The Image Foresting Transform: Theory, Algorithms, and Applications

Alexandre X. Falcão, Jorge Stolfi, and Roberto de Alencar Lotufo

Abstract—The image foresting transform (IFT) is a graph-based approach to the design of image processing operators based on connectivity. It naturally leads to correct and efficient implementations and to a better understanding of how different operators relate to each other. We give here a precise definition of the IFT, and a procedure to compute it—a generalization of Dijkstra’s algorithm—with a proof of correctness. We also discuss implementation issues and illustrate the use of the IFT in a few applications.

Index Terms—Dijkstra’s algorithm, shortest-path problems, image segmentation, image analysis, regional minima, watershed transform, morphological reconstruction, boundary tracking, distance transforms, and multiscale skeletonization.

1 INTRODUCTION

THE *image foresting transform* (IFT), which we describe here, is a general tool for the design, implementation, and evaluation of image processing operators based on connectivity. The IFT defines a *minimum-cost path forest* in a graph, whose nodes are the image pixels and whose arcs are defined by an *adjacency relation* between pixels. The cost of a path in this graph is determined by an application-specific *path-cost function*, which usually depends on local image properties along the path—such as color, gradient, and pixel position. The roots of the forest are drawn from a given set of *seed pixels*. For suitable path-cost functions, the IFT assigns one minimum-cost path from the seed set to each pixel, in such a way that the union of those paths is an oriented forest, spanning the whole image. The IFT outputs three attributes for each pixel: its predecessor in the optimum path, the cost of that path, and the corresponding root (or some label associated with it). A great variety of powerful image operators, old and new, can be implemented by simple local processing of these attributes.

We describe a general algorithm for computing the IFT, which is essentially Dijkstra’s shortest-path algorithm [1], [2], [3], slightly modified for multiple sources and general path-cost functions. Since our conditions on the path costs apply only to optimum paths, and not to all paths, we found it necessary to rewrite and extend the classical proof of correctness of Dijkstra’s algorithm. In many practical applications, the path costs are integers with limited increments and the graph is sparse; therefore, the optimizations described by Dial [4] and Ahuja et al. [5] will apply and the running time will be linear on the number of pixels.

The IFT unifies and extends many image analysis techniques which, even though based on similar underlying concepts (ordered propagation, flooding, geodesic dilations,

dynamic programming, region growing, A^* graph search, etc.), are usually presented as unrelated methods. Those techniques can all be reduced to a partition of the image into *influence zones* associated with a given seed set, where the zone of each seed consists of the pixels that are “more closely connected” to that seed than to any other, in some appropriate sense. These influence zones are simply the trees of the forest defined by the IFT. Examples are watershed transforms [6], [7] and fuzzy-connected segmentation [8], [9], [10], [11]. The IFT also provides a mathematically sound framework for many image-processing operations that are not obviously related to image partition, such as morphological reconstruction [12], distance transforms [13], [14], multiscale skeletons [14], shape saliences and multiscale fractal dimension [15], [16], and boundary tracking [17], [18], [19].

By separating the general forest computation procedure from the application-specific path-cost function, the IFT greatly simplifies the implementation of image operators, and provides a fair testbed for their evaluation and tuning. For many classical operators, the IFT-based implementation [6] is much closer to the theoretical definition than the published algorithms [20], [21]. Indeed, many algorithms which have been used without proof [20], [21], [22], [23], [24] have their correctness established by being reformulated in terms of the IFT [6], [12]. By clarifying the relationship between different image transforms [12], the IFT approach often leads to novel image operators and considerably faster (but still provably correct) algorithms [14], [25], [26].

The IFT definition and algorithms are independent of the nature of the pixels and of the dimension of the image, and therefore they apply to color and multispectral images, as well as to higher-dimensional images such as video sequences and tomography data [26].

Section 2 reviews the previous related work. We define the IFT and related concepts in Section 3. Section 4 presents the basic IFT algorithm, a proof of its correctness and optimization hints. Tie-breaking policies are discussed in Section 5. In Section 6, we illustrate the use of the IFT in a few selected applications. Section 7 contains the conclusions and current research on the IFT.

- A.X. Falcão and J. Stolfi are with the Institute of Computing, University of Campinas, Av. Albert Einstein, 1251, CEP 13084-851, Campinas, SP, Brasil. E-mail: {afalcão, stolfi}@ic.unicamp.br.
- R. de Alencar Lotufo is with the Faculty of Electrical and Computing Engineering, University of Campinas, Av. Albert Einstein, 400, CEP 13083-970, Campinas, SP, Brasil. E-mail: lotufo@dca.fee.unicamp.br.

Manuscript received 17 May 2002; revised 27 Mar. 2003; accepted 3 July 2003.
Recommended for acceptance by E. Hancock.
For information on obtaining reprints of this article, please send e-mail to: tpami@computer.org, and reference IEEECS Log Number 116572.

2 RELATED WORK

2.1 Shortest-Path Problems in General Graphs

The problem of computing shortest paths in a general graph has an enormous bibliography [27]. Most authors dispose of the multiple-source problem by trivial reduction to the single-source version. Solutions to the latter were first proposed by Moore in 1957 [1] and Bellman in 1958 [2]. The running time of their algorithms was $O(mn)$, in the worst case, for a graph with n nodes and m arcs. An improved algorithm was given by Dijkstra in 1959 [3]; its running time of $O(n^2)$ can be reduced to $O(m + n \log n)$ through the use of a balanced heap data structure [5]. In 1969, Dial published a variant of Dijkstra's algorithm [4], [5], for the special case where the arc costs are integers in the range $[0..K]$, which uses bucket sorting to achieve running time $O(m + nK)$.

The standard path-cost function is the sum of arc costs. Extensions to more general cost models fall in two major classes. The algebraic or "semiring" model considers fixed arc costs that are combined by an associative operation, that distributes over the min function (or, more generally, over a second associative operator which is used to summarize all paths leading to a given node) [28], [29]. Another model does not assume associativity or fixed arc costs, but requires that the cost of a path be computed incrementally from the cost of its prefixes through a composition function that satisfies certain monotonicity constraints [30]. We note that, in both approaches, the constraints on the path-cost functions are required to hold for all paths, not just for optimum ones.

2.2 Image Processing Using Graph Concepts

In image processing, Montanari [31] and Martelli [32], [33] were the first to formulate a boundary finding problem as a shortest-path problem in a graph. Montanari [31] required the boundary to be star-shaped [34]. Martelli [32], [33] considered only a selected subset of the arcs and, therefore, his algorithm may fail in some situations. These restrictions have since been lifted by Falcão et al. [17] and Mortensen and Barrett [35]. In region-based image segmentation, Udupa and Samarasekera [8], and Saha and Udupa [9] proposed the fuzzy connectedness theory for object definition, which was efficiently implemented by Nyúl et al. using Dial's bucket queue [10]. Verwer et al. [36] and Sharaiha and Christofides [37] used the Dial's algorithm to compute weighted distance and chamfer distance transforms. Meyer extended their work to some variations of the watershed transform based on the eikonal equation [38]. Dial's bucket queue became the core of fast ordered propagation algorithms for various applications, including Euclidean distance transform [39], [40], watershed transforms [20], [21], [41], and morphological reconstructions [22], [23], [24].

2.3 The PDE Approach

There is a certain formal resemblance between the IFT and the *partial differential equations* (PDE) approach to image processing [42], since both involve the sweeping of the image by a propagating front—which, in the IFT approach, is the boundary between the sets S and $\bar{S}$ of Dijkstra's original algorithm [5]. The eikonal problem, specifically, asks for the minimum-time path from the initial front to each point of the domain. The reciprocal of the local speed of propagation is analogous to the arc costs of the IFT model, and the Fast Marching method is seen as the analog of Dijkstra's algorithm [43].

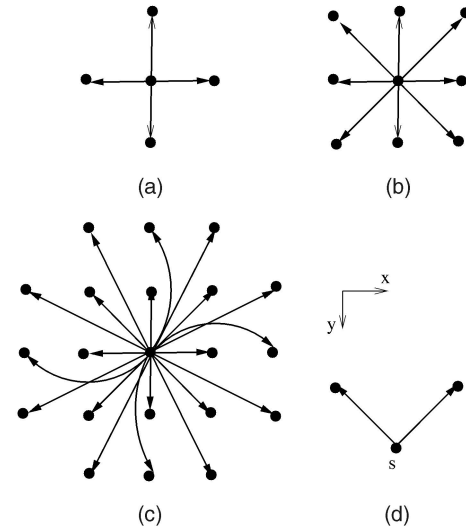


Fig. 1. (a), (b), and (c) Euclidean adjacency relations for $\rho = 1, \sqrt{2}$ and $\sqrt{5}$, respectively. (d) An asymmetric adjacency relation given by $\mathcal{M} = \{(-1, -1), (1, -1)\}$.

To a large extent, the comparison of the two approaches is just another instance of the continuous versus discrete question. The discreteness of the IFT allows rigorous proofs and predictable algorithms, free from approximation errors and numerical instabilities that often affect the PDE. The IFT model also allows the cost of extending a path to depend on the whole path and not just on its current cost. On the other hand, the PDE approach allows the propagation front to move back and forth, or with speed that depends on its local shape—features that are used in contour smoothing, contour detection, and other applications. It is not obvious whether similar effects can be achieved in the IFT model, where the "shape of the front" is not a well-defined concept.

3 NOTATION AND DEFINITIONS

An *image* $\mathbf{I}$ is a pair $(\mathcal{I}, I)$ consisting of a finite set $\mathcal{I}$ of *pixels* (points in $\mathbb{Z}^2$), and a mapping I that assigns to each pixel t in $\mathcal{I}$ a *pixel value* $I(t)$ in some arbitrary value space.

An *adjacency relation* $\mathcal{A}$ is an irreflexive binary relation between pixels of $\mathcal{I}$. Once the adjacency relation $\mathcal{A}$ has been fixed, the image $\mathbf{I}$ can be interpreted as a directed graph whose nodes are the image pixels and whose arcs are the pixel pairs in $\mathcal{A}$. In what follows, $n = |\mathcal{I}|$ is the number of nodes (pixels) and $m = |\mathcal{A}|$ is the number of arcs.

In most image-processing applications, the adjacency relation $\mathcal{A}$ is *translation-invariant*, meaning that sAt depends only on the relative position $t - s$ of the pixels. For example, one often takes $\mathcal{A}$ to consist of all pairs of distinct pixels $(s, t) \in \mathcal{I} \times \mathcal{I}$ such that $d(s, t) \leq \rho$, where $d(s, t)$ denotes the Euclidean distance and ρ is a specified constant. Figs. 1a, 1b, and 1c show the adjacent pixels of a fixed central pixel s in this *Euclidean adjacency relation*, when $\rho = 1$ (the 4-connected adjacency), $\rho = \sqrt{2}$ (8-connected), and $\rho = \sqrt{5}$, respectively.

More generally, we can take a finite subset $\mathcal{M}$ of $\mathbb{Z}^2 \setminus \{(0, 0)\}$; for example, $\mathcal{M} = \{(-1, -1), (1, -1)\}$ and define $\mathcal{A}$ as all pairs of pixels (s, t) , where $t - s \in \mathcal{M}$. See Fig. 1d. Note that the adjacency relation does not have to be symmetric.

A *path* is a sequence of pixels $\pi = \langle t_1, t_2, \dots, t_k \rangle$, where $(t_i, t_{i+1}) \in \mathcal{A}$ for $1 \leq i \leq k - 1$. We denote the *origin* t_1 and

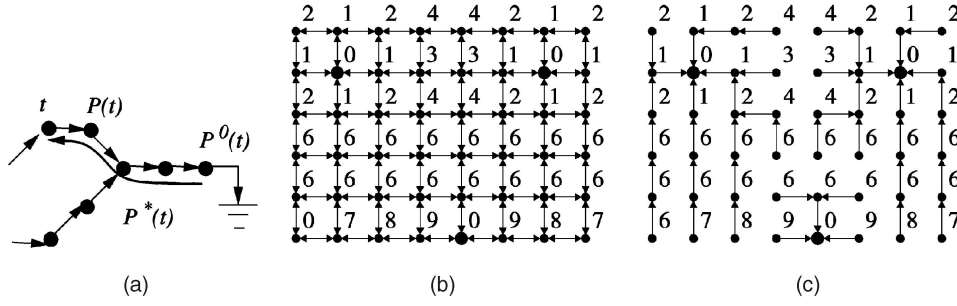


Fig. 2. (a) The main elements in a spanning forest. (b) An image graph with 4-connected adjacency, where the integers are the image values $I(t)$. (c) An optimum-path forest for the path-cost function $f_{\max}$, where $h(t) = w(s, t) = I(t)$, restricted to the three seed pixels represented by bigger dots.

the destination t_k of π by $\text{org}(\pi)$ and $\text{dst}(\pi)$, respectively. The path is *trivial* if $k = 1$. If π and τ are paths such that $\text{dst}(\pi) = \text{org}(\tau) = t$, we denote by $\pi \cdot \tau$ the concatenation of the two paths, with the two joining instances of t merged into one.

3.1 Path Costs

We assume given a function f that assigns to each path π a *path cost* $f(\pi)$, in some totally ordered set $\mathcal{V}$ of cost values. Without loss of generality, we can always assume that $\mathcal{V}$ contains a maximum element, which we denote by $+\infty$. Usually, the path cost depends on local properties of the image $\mathbf{I}$ —such as color, gradient, and pixel position—along the path.

A popular example is the *additive* path-cost function, which satisfies

$$\begin{aligned} f_{\text{sum}}(\langle t \rangle) &= h(t), \\ f_{\text{sum}}(\pi \cdot \langle s, t \rangle) &= f_{\text{sum}}(\pi) + w(s, t), \end{aligned} \quad (1)$$

where $(s, t) \in \mathcal{A}$, π is any path ending at s , $h(t)$ is a fixed *handicap cost* for any paths starting at pixel t , and $w(s, t)$ is a fixed nonnegative weight assigned to the arc (s, t) .

Another important example is the *max-arc* path-cost function $f_{\max}$, defined by

$$\begin{aligned} f_{\max}(\langle t \rangle) &= h(t), \\ f_{\max}(\pi \cdot \langle s, t \rangle) &= \max\{f_{\max}(\pi), w(s, t)\}, \end{aligned} \quad (2)$$

where $h(t)$ and $w(s, t)$ are fixed but arbitrary handicap and arc weight functions. Note that f_{sum} and $f_{\max}$ are distinct models: Since the incremental cost $f_{\max}(\pi \cdot \langle s, t \rangle) - f_{\max}(\pi)$ depends on the cost of π , one cannot, in general, redefine $f_{\max}$ as the sum of *fixed* weights $w'(s, t)$.

3.2 Seed Pixels

In typical applications of the IFT, we would like to use a predefined path-cost function f but constrain the search to paths that start in a given set $\mathcal{S} \subseteq \mathcal{I}$ of *seed pixels*. We can model this constraint by defining a new path-cost function $f^{\mathcal{S}}(\pi)$, which is equal to $f(\pi)$ when $\text{org}(\pi) \in \mathcal{S}$, and $+\infty$ otherwise. In particular, for the f_{sum} and $f_{\max}$ functions, this is equivalent to setting $h(t) = +\infty$ for pixels $t \notin \mathcal{S}$.

3.3 Optimum Paths

We say that a path π is *optimum* if $f(\pi) \leq f(\pi')$ for any other path π' with $\text{dst}(\pi') = \text{dst}(\pi)$, irrespective of its starting point. In that case, $f(\pi)$ is by definition the *cost* of pixel $t = \text{dst}(\pi)$, denoted by $f(t)$. Note that a trivial path is not necessarily optimum: even for f_{sum} or $f_{\max}$, the handicaps of two pixels s and t may be such that a nontrivial path from s to t is cheaper than the trivial path $\langle t \rangle$.

3.4 Spanning Forests

A *predecessor map* is a function P that assigns to each pixel t in $\mathcal{I}$ either some other pixel in $\mathcal{I}$, or a distinctive marker *nil* not in $\mathcal{I}$ —in which case t is said to be a *root* of the map. A *spanning forest* is a predecessor map which contains no cycles—in other words, one which takes every pixel to *nil* in a finite number of iterations. For any pixel $t \in \mathcal{I}$, a spanning forest P defines a path $P^*(t)$ recursively as $\langle t \rangle$ if $P(t) = \text{nil}$, and $P^*(s) \cdot \langle s, t \rangle$ if $P(t) = s \neq \text{nil}$. We will denote by $P^0(t)$ the initial pixel of $P^*(t)$ (see Fig. 2a).

3.5 The Image Foresting Transform

The *image foresting transform* (IFT) takes an image $\mathbf{I}$, a path-cost function f and an adjacency relation $\mathcal{A}$, and returns an *optimum-path forest*—a spanning forest P such that $P^*(t)$ is optimum, for every pixel t . See Figs. 2b and 2c.

Note that, in an optimum-path forest P for a seed-restricted cost function $f^{\mathcal{S}}$, any pixel t with finite cost $f^{\mathcal{S}}(P^*(t))$ will belong to a tree whose root is a seed pixel; however, some seeds may not be roots of the forest because they may be more cheaply sourced by another root seed.

In general, there may be many paths of minimum cost leading to a given pixel; only the pixel costs $f(t)$ are uniquely defined. Observe that, if we independently pick an optimum path for each pixel, the union of those paths may not be a forest. Indeed, certain graphs and cost functions may not even admit *any* optimum-path forest. Sufficient conditions for the existence of the IFT will be given in Section 4.3.

4 ALGORITHM

For suitable path-cost functions, the IFT can be computed by Algorithm 1 below—which is essentially Dijkstra's procedure for computing minimum-cost paths from a single source in a graph [3], [5], slightly modified to allow multiple sources and more general cost functions. This variant was chosen for maximum flexibility and to simplify the proof of correctness. In practice, Algorithm 1 can be optimized in a number of ways (see Section 4.5).

Algorithm 1. *Input:* An image $\mathbf{I} = (\mathcal{I}, I)$; an adjacency relation $\mathcal{A} \subset \mathcal{I} \times \mathcal{I}$; and a path-cost function f . *Output:* An optimum-path forest P . *Auxiliary Data Structures:* Two sets of pixels $\mathcal{F}, \mathcal{Q}$ whose union is $\mathcal{I}$.

1. Set $\mathcal{F} \leftarrow \{\}$, $\mathcal{Q} \leftarrow \mathcal{I}$. For all $t \in \mathcal{I}$, set $P(t) \leftarrow \text{nil}$.
2. While $\mathcal{Q} \neq \{\}$, do
 - 2.1. Remove from $\mathcal{Q}$ a pixel s such that $f(P^*(s))$ is minimum, and add it to $\mathcal{F}$.
 - 2.2. For each pixel t such that $(s, t) \in \mathcal{A}$, do
 - 2.2.1. If $f(P^*(s) \cdot \langle s, t \rangle) < f(P^*(t))$, set $P(t) \leftarrow s$.

4.1 General Properties

Some basic facts about Algorithm 1 are easily established by induction on the number of steps. Since every iteration of the main loop removes from $\mathcal{Q}$ exactly one pixel (which is never returned) and each arc of $\mathcal{A}$ is examined exactly once in Step 2.2, it follows that

Lemma 1. *Algorithm 1 terminates in $O(n)$ iterations of the outer loop, and $O(m)$ total iterations of the inner loop.*

Moreover, each predecessor $P(t)$ is initially *nil* and is modified only by the assignment $P(t) \leftarrow s$ in Step 2.2.1—when t is still in $\mathcal{Q}$ but s is in $\mathcal{F}$. Lemma 2 follows

Lemma 2. *The predecessor map P computed by Algorithm 1 is always a spanning forest.*

Note that Lemmas 1 and 2 hold for *any* path-cost function f . This is more of a curse than a blessing, because it tempts people into using Algorithm 1 even when its result is not an optimum-path forest. The optimality depends on f being sufficiently well-behaved.

4.2 Monotonic-Incremental Cost Functions

When the path-cost function is additive, the correctness of Algorithm 1 is established by the standard proof of Dijkstra's method [28], [5]. Note that the extension to multiple starting pixels is trivial since it is equivalent to adding extra arcs from a dummy starting pixel $u \notin \mathcal{I}$ to all pixels in $\mathcal{I}$ and setting $w(u, t) = h(t)$ for each new arc (u, t) . This remains true even when the arc costs and handicaps are allowed to be $+\infty$.

In fact, as shown by Frieze [30], the original proof of Dijkstra's algorithm is easily generalized to *monotonic-incremental* (MI) path-cost functions, which satisfy

$$\begin{aligned} f(\langle t \rangle) &= h(t), \\ f(\pi \cdot \langle s, t \rangle) &= f(\pi) \odot (s, t), \end{aligned} \quad (3)$$

where $h(t)$ is an arbitrary handicap cost, and $\odot : \mathcal{V} \times \mathcal{A} \rightarrow \mathcal{V}$ is a binary operation that satisfies the conditions

$$\text{M1. } x' \geq x \Rightarrow x' \odot (s, t) \geq x \odot (s, t),$$

$$\text{M2. } x \odot (s, t) \geq x,$$

for any $x, x' \in \mathcal{V}$ and any $(s, t) \in \mathcal{A}$. An essential feature of this model is that $\odot$ depends only on the *cost* of π , and not on any other property of π . Both the additive cost f_{sum} (with nonnegative arc weights) and the max-arc cost f_{max} are monotonic-incremental. It turns out that most image processing problems that we have successfully reduced to the IFT require MI path-cost functions, and therefore can be solved by Algorithm 1. In fact, Condition M2 can be weakened to $f(\pi \cdot \kappa) \geq f(\pi)$ for any cycle κ [30].

On the other hand, it is easy to find counterexamples of path-cost functions—not MI, of course—which cause Algorithm 1 to fail. A textbook counterexample is the additive path-cost f_{sum} when the arc weights $w(s, t)$ are allowed to be negative. The algorithm may also fail for generalizations of f_{sum} or f_{max} where the arc weight $w(s, t)$ is allowed to depend on the path already chosen for s . Unfortunately, this applies to several path-cost functions that would seem reasonable for image processing. For example, in multi-seeded region-based segmentation, it may seem reasonable to use the function $f_{\text{abs}}(\pi)$, defined as the maximum of $|I(t) - I(\text{org}(\pi))|$ for any pixel t along the path π . Fig. 3 shows a situation where Dijkstra's algorithm will fail (and, in fact, where the optimum paths do not form a forest). The

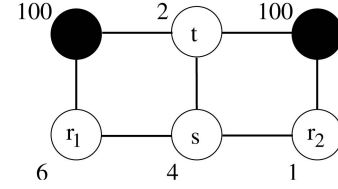


Fig. 3. A 2×3 image where Algorithm 1 fails to produce an optimum path forest for f_{abs}^S , with $S = \{r_1, r_2\}$. We have $\hat{f}_{\text{abs}}^S(s) = f_{\text{abs}}(\langle r_1, s \rangle) = 2 < f_{\text{abs}}(\langle r_2, s \rangle) = 3$, but $\hat{f}_{\text{abs}}^S(t) = f_{\text{abs}}(\langle r_2, s, t \rangle) = 3 < f_{\text{abs}}(\langle r_1, s, t \rangle) = 4$.

same image graph is a counterexample when the path cost $f(\pi)$ is defined as the variance of the pixel values along π . Another counterexample is the region-growing criterion proposed by Bischof and Adams [44], where each candidate pixel is ranked by the absolute difference between its value and the mean value of all pixels in each region.

4.3 Smooth Path-Cost Functions

Algorithm 1 *does* work for certain cost functions which are not MI, or even monotonic. An example is the 4-connected adjacency and the function $f_{\text{euc}}^S(\pi)$ defined as the Euclidean distance between the endpoints of π , restricted to paths that start at a given seed set S . Algorithm 1 will correctly find an optimum-path forest as long as $|S| \leq 2$, even though Condition M2 is violated when $|S| = 1$, and both M1 and M2 are violated when $|S| = 2$. Consider, for instance, three pixels $t = r_1 = (0, 0)$, $s = (2, 0)$, $r_2 = (3, 0)$: We have $f_{\text{euc}}(\langle r_1, s \rangle) > f_{\text{euc}}(\langle r_2, s \rangle)$, but $f_{\text{euc}}(\langle r_1, s \rangle \cdot \langle s, t \rangle) < f_{\text{euc}}(\langle r_2, s \rangle \cdot \langle s, t \rangle)$; and, also, $f_{\text{euc}}(\langle s, t \rangle \cdot \langle t, s \rangle) < f_{\text{euc}}(\langle s, t \rangle)$. (The algorithm may fail, however, if $|S| \geq 3$; see Section 6.4.)

Examples like this one led us to search for conditions on the path-cost function f that are more general than M1 and M2 but still strong enough to ensure the correctness of Algorithm 1. Specifically, we claim that the algorithm will work if, for any pixel $t \in \mathcal{I}$, there is an optimum path π ending at t which either is trivial, or has the form $\tau \cdot \langle s, t \rangle$, where

$$\text{C1. } f(\tau) \leq f(\pi),$$

$$\text{C2. } \tau \text{ is optimum, and}$$

$$\text{C3. for any optimum path } \tau' \text{ ending at } s, f(\tau' \cdot \langle s, t \rangle) = f(\pi).$$

These conditions seem to capture the essential features of the path-cost function that are used in the classical proofs (cf. Bellman's optimality principle [45]). Observe that Conditions C1, C2, and C3 are not required to hold for *all* paths ending at t , but only for *some* path π that is optimum. We say that a path-cost function f satisfying Conditions C1, C2, and C3 is *smooth*.

It can be checked that any MI path-cost function satisfies Conditions C1, C2, and C3. Also, if f is an MI cost function, then its restriction f^S to an arbitrary seed set S will be MI, and hence smooth too. (Unfortunately, this is not necessarily true if f is smooth, but not MI.)

For an example of a smooth function that is not MI, let f be an MI function and define $f'(\pi) = f(\pi) + g(\pi)$, where $g(\pi)$ is zero if π is optimum for f , and an arbitrary positive value otherwise. The function f' satisfies Conditions C1, C2, and C3, even though it may fail M1 for arbitrary paths. For a more realistic example, it can be checked that f_{euc}^S is smooth when $|S| \leq 2$. Two key observations are that the influence zones of the seed pixels are 4-connected and that any path π with minimum number of arcs connecting S to t satisfies Conditions C1, C2, and C3.

4.4 Proof of Correctness

In order to prove the correctness of Algorithm 1 for a smooth path-cost function f , we require some auxiliary definitions. Let $\mathcal{B} \subseteq \mathcal{A}$ denote the set of all arcs of $\mathcal{A}$ that have already been considered in Step 2.2. We say that a path is a $(\mathcal{B}, P)$ -path if it is either a trivial path $\langle t \rangle$ with $t \in \mathcal{Q}$ or has the form $P^*(s) \cdot \langle s, t \rangle$, where $s \in \mathcal{F}$, $t \in \mathcal{Q}$, and (s, t) is an arc of $\mathcal{B}$. We also say that a path is a $(\mathcal{B}, P)$ -optimum if it is a $(\mathcal{B}, P)$ -path, and has minimum cost among all $(\mathcal{B}, P)$ -paths with the same final pixel. Note that a $(\mathcal{B}, P)$ -optimum path may not be optimum overall and, therefore, we cannot assume that it satisfies Conditions C1, C2, and C3.

The proof is based on the following invariants that, we claim, hold every time the algorithm reaches one of the Steps 2.1, 2.2, or 2.2.1:

- H1. For every pixel $s \in \mathcal{F}$, the path $P^*(s)$ is optimum.
- H2. For every pixel $s \in \mathcal{F}$, and any path π that ends in $\mathcal{Q}$, we have $f(P^*(s)) \leq f(\pi)$.
- H3. For every pixel $t \in \mathcal{Q}$, the path $P^*(t)$ is $(\mathcal{B}, P)$ -optimum.

The bulk of the proof lies in Lemmas 3 and 4 below, which are combined in Theorem 1. The proofs of these lemmas are given in Appendix A.

Lemma 3. *Each execution of Step 2.2.1 preserves Invariants H1, H2, and H3.*

Lemma 4. *If f is a smooth path-cost function, Step 2.1 preserves Invariants H1, H2, and H3.*

Theorem 1. *Algorithm 1 computes an optimum-path forest for any smooth cost function f .*

Proof. Invariants H1, H2, and H3 are trivially valid just before Step 2. By Lemmas 3 and 4, they remain valid through the execution of the algorithm, which terminates by Lemma 1. Upon termination, P is a spanning forest by Lemma 2 and $\mathcal{F} = \mathcal{I}$ together with Invariant H1 imply that all paths $P^*(t)$ are optimum. $\square$

4.5 Efficient Implementation

Algorithm 1 can be optimized in a number of ways, without affecting its correctness:

Reusing the path costs. One can avoid recomputing $f(P^*(t))$ at every iteration of Step 2.2.1 by storing $f(P^*(t))$ in a table $C(t)$. Moreover, the cost $f(P^*(s) \cdot \langle s, t \rangle)$ can often be computed in $O(1)$ time from $w(s, t)$ and $f(P^*(s)) = C(s)$ (and, if f is not MI, from some other accumulated information about $P^*(s)$). At the end of the algorithm, $C(t)$ will be the minimum pixel cost $\hat{f}(t)$.

Avoiding backward arcs. From Invariants H1 and H2, it follows that Step 2.2.1 may have an effect only if the current cost $f(P^*(t))$ of t is strictly higher than $f(P^*(s))$. Therefore, we can avoid computing the cost $f(P^*(s) \cdot \langle s, t \rangle)$ whenever $C(t) \leq C(s)$.

Handling of infinite costs. In many applications, the cost $f(P^*(t))$ is $+\infty$ for most pixels $t \in \mathcal{Q}$, through most of the execution of Algorithm 1. In such cases, we can save a large fraction of the running time by storing in $\mathcal{Q}$ only those pixels $t \in \mathcal{I} \setminus \mathcal{F}$ which have $f(P^*(t)) < +\infty$. In particular, if $f = f^S$ for some set S , then Step 1 should initialize $\mathcal{Q}$ with S rather than $\mathcal{I}$. With this modification, any pixel t that is not reachable by any finite-cost path (in particular, any pixel which is not reachable from S) will remain a single-node tree.

Infinite costs are also useful when we are only interested in paths whose costs do not exceed a specified threshold $C_{\max}$. In those cases, we can modify f by mapping any cost greater than $C_{\max}$ to $+\infty$. Note that the modified algorithm will terminate after enumerating all pixels t with finite cost $\hat{f}(t)$.

Efficient pixel queue structure. Asymptotically, the bottleneck of Algorithm 1 lies in Step 2.1, the selection of the minimum-cost pixel $s \in \mathcal{Q}$. If $\mathcal{Q}$ is implemented as a balanced heap data structure, the total running time will be $O(m + n \log n)$.

In most applications, the path costs $f(\pi)$ are either $+\infty$ or integers in some range $[C_{\min} .. C_{\max}]$. In such cases, we can implement $\mathcal{Q}$ as a vector of C_{Δ} buckets, each pointing to a circular doubly linked list of pixels, where $C_{\Delta} = C_{\max} - C_{\min} + 1$. The insertion, deletion, and cost update of a pixel can then be done in $O(1)$ time. Finding the next minimum-cost pixel in $\mathcal{Q}$ may require skipping over several empty buckets; however, since the cost of the next pixel never decreases, the total time for Step 2.1 is $O(C_{\Delta})$. Algorithm 1 will then run in $O(m + n + C_{\Delta})$ time and $O(n + C_{\Delta})$ storage. Note that each pixel may appear in the queue at most once; and so we can avoid dynamic allocation by preallocating two links $\text{next}(p)$ and $\text{prev}(p)$ for each pixel p , which are then used to connect the pixels belonging to each bucket [18].

Moreover, in many of those applications, there exists a fairly small upper bound K to the incremental cost $f(\pi \cdot \langle s, t \rangle) - f(\pi)$ of extending an optimum path π by an arc $(s, t) \in \mathcal{A}$, and to the maximum difference $f(\langle t \rangle) - f(\langle t' \rangle)$ between the costs of trivial paths (excluding infinite values). Invariant H3 then implies that, for all $t \in \mathcal{Q}$, the cost $f(P^*(t))$ is either $+\infty$ or an integer in the range $[C .. C + K]$, for some cost C that varies during the algorithm. The bucket vector can then be replaced by a circular queue of $K + 1$ entries, reducing the storage cost to $O(n + K)$ [4], [5].

Furthermore, C_{Δ} will be bounded by K for $f_{\max}$ and by nK for general smooth cost functions, in the worst case. (In practice, an optimum path is unlikely to traverse a significant fraction of the image pixels, so its cost is usually bounded by a few times $K\sqrt{n}$.) Therefore, the algorithm will run in $O(m + nK)$ time. As noted by Ahuja et al. [5], there are more complicated data structures that achieve asymptotic cost $O(m + n\sqrt{\log K})$. However, for practical values of n , K , and the constants hidden in the O notation, it is not always the case that the bucket-based queue will be more efficient than a balanced heap [10].

Propagating the root labels. Although not needed by the algorithm itself, many applications (and some path-cost functions) need to know the root pixel $P^0(t)$ associated with each pixel t —or, more generally, some attribute $\lambda(P^0(t))$ that was assigned to it. For example, in segmentation applications, one would assign the same label to all seed pixels that belong to the same target object, and treat the corresponding trees as a single region.

The root labels $\lambda(P^0(t))$ can be computed in linear time from the predecessor map P , for all $t \in \mathcal{I}$, as a post-processing step; but it is more convenient to compute them during Algorithm 1, as a *root label map* $L(t)$.

Optimized Algorithm. All these improvements are implemented in Algorithm 2 below:

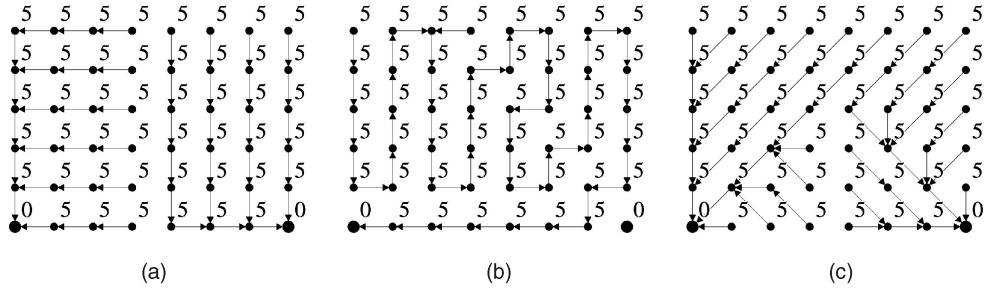


Fig. 4. Three examples of tie-breaking for the same function of Fig. 2, restricted to the two seed pixels represented by bigger dots. (a) FIFO policy and 4-connected adjacency. (b) LIFO policy and 4-connected adjacency. (c) FIFO policy and 8-connected adjacency.

Algorithm 2. *Input:* An image $\mathbf{I} = (\mathcal{I}, I)$; an adjacency relation $\mathcal{A} \subset \mathcal{I} \times \mathcal{I}$; a labeling function λ defined on $\mathcal{I}$; and a smooth path-cost function f . *Output:* An optimum-path forest P , and the corresponding cost map C and root label map L . *Auxiliary Data Structures:* A priority queue $\mathcal{Q}$.

1. For all $t \in \mathcal{I}$, set $P(t) \leftarrow \text{nil}$, $L(t) \leftarrow \lambda(t)$, and $C(t) \leftarrow f(\langle t \rangle)$. If $C(t) < +\infty$, insert t in $\mathcal{Q}$.
2. While $\mathcal{Q}$ is not empty, do
 - 2.1. Remove from $\mathcal{Q}$ a pixel s such that $C(s)$ is minimum.
 - 2.2. For each t such that $(s, t) \in \mathcal{A}$ and $C(t) > C(s)$, do
 - 2.2.1. Compute $C' = f(P^*(s) \cdot \langle s, t \rangle)$.
 - 2.2.2. If $C' < C(t)$, then
 - 2.2.2.1. If $C(t) \neq +\infty$, remove t from $\mathcal{Q}$.
 - 2.2.2.2. Set $P(t) \leftarrow s$, $C(t) \leftarrow C'$, $L(t) \leftarrow L(s)$, and insert t in $\mathcal{Q}$.

5 TIE-BREAKING

As observed in Section 3.5, the optimum-path forest is not always unique. Since path costs are usually discrete, multiple solutions are actually quite common in practice.

Algorithms 1 and 2 already resolve some of those ambiguities. For instance, when a pixel t is reached by two optimum paths of equal cost, the algorithms will set $P^*(t)$ to the path that is found first. The only remaining ambiguity is the choice of the minimum-cost pixel s in $\mathcal{Q}$, in case of ties. Usually, the most convenient choice is to pick the pixel that entered in $\mathcal{Q}$ first, i.e., ties are broken by using the *first-in first-out* (FIFO) policy.

In some cases, however, a *last-in first-out* (LIFO) policy may be more adequate. See Algorithm 3 below. For consistency, in this case, the algorithm should also choose always the *last* optimum path found in case of ties.

Algorithm 3. *Input:* An image $\mathbf{I} = (\mathcal{I}, I)$; an adjacency relation $\mathcal{A} \subset \mathcal{I} \times \mathcal{I}$; a labeling function λ defined on $\mathcal{I}$; and a smooth path-cost function f . *Output:* An optimum-path forest P , and the corresponding cost map C and root label map L . *Auxiliary Data Structures:* A priority queue $\mathcal{Q}$, with LIFO tie-breaking.

1. For all $t \in \mathcal{I}$, set $P(t) \leftarrow \text{nil}$, $L(t) \leftarrow \lambda(t)$, $C(t) \leftarrow f(\langle t \rangle)$, and insert t in $\mathcal{Q}$.
2. While $\mathcal{Q}$ is not empty, do
 - 2.1. Remove from $\mathcal{Q}$ a pixel s such that $C(s)$ is minimum.
 - 2.2. For each pixel t such that $(s, t) \in \mathcal{A}$ and $t \in \mathcal{Q}$, do
 - 2.2.1. Compute $C' = f(P^*(s) \cdot \langle s, t \rangle)$.
 - 2.2.2. If $C' \leq C(t)$, remove t from $\mathcal{Q}$, set $P(t) \leftarrow s$, $C(t) \leftarrow C'$, $L(t) \leftarrow L(s)$, and insert t in $\mathcal{Q}$.

The main difference between Algorithms 2 and 3, besides the queue policy, lies in Step 2.2.2—where $P(t)$ and $L(t)$ must be updated, and t must be removed and reinserted in $\mathcal{Q}$, even when C' is equal to $C(t)$.

With the FIFO policy, any connected set of pixels that could be reached from two or more roots, at the same cost, will tend to get partitioned among the respective trees (see Fig. 4a). With the LIFO policy, in contrast, those pixels will all get assigned to a single tree (see Fig. 4b). While the LIFO policy may be an advantage in some applications (see Section 6.1), in most cases the shorter paths and more uniform trees produced by the FIFO policy are a better match to the users' expectations.

It must be noted, however, that the FIFO policy by itself does not ensure an unbiased partition. In the extreme case where any path from any given seed has the same cost, one might expect that the boundary between adjacent trees would follow the bisector of their roots, in the graph-theoretic metric (Fig. 4a). In fact, one of the trees usually encroaches into the other, so the boundary may deviate quite sharply from the bisector (see Fig. 4c). Thus, when the partition of ambiguous pixels is important, one must incorporate an appropriate tie-breaking criterion explicitly into the path-cost function.

6 APPLICATIONS

In this section, we illustrate a few selected applications of the IFT. Even though the examples given here use gray-scale images, many of these operators can be trivially applied to multispectral (color) images, by changing the path-cost function to use all bands.

6.1 Regional Minima

A *regional minimum* is a maximal connected set $\mathcal{X} \subseteq \mathcal{I}$, such that $I(s) \leq I(t)$ for any $s \in \mathcal{X}$ and any arc $(s, t) \in \mathcal{A}$ [23]. We can compute the regional minima with the cost function

$$f_{\text{ini}}(\langle t \rangle) = I(t), \text{ for all } t \in \mathcal{I},$$

$$f_{\text{ini}}(\pi \cdot \langle s, t \rangle) = \begin{cases} f_{\text{ini}}(\pi), & \text{if } I(s) \leq I(t), \\ +\infty, & \text{otherwise.} \end{cases} \quad (4)$$

The function f_{ini} is MI, and therefore smooth. Note that any optimum path starts at a regional minimum and never goes downhill. Since the regional minima are contained in the set $\mathcal{S}$ of pixels s such that $I(s) \leq I(t)$ for all $(s, t) \in \mathcal{A}$, we can use $f_{\text{ini}}^{\mathcal{S}}$ instead of f_{ini} .

With FIFO tie-breaking, a pixel will be a root of the IFT if and only if it belongs to a regional minimum and each regional minimum is a connected component of the set of all roots. With LIFO tie-breaking, in contrast, we will get *exactly*

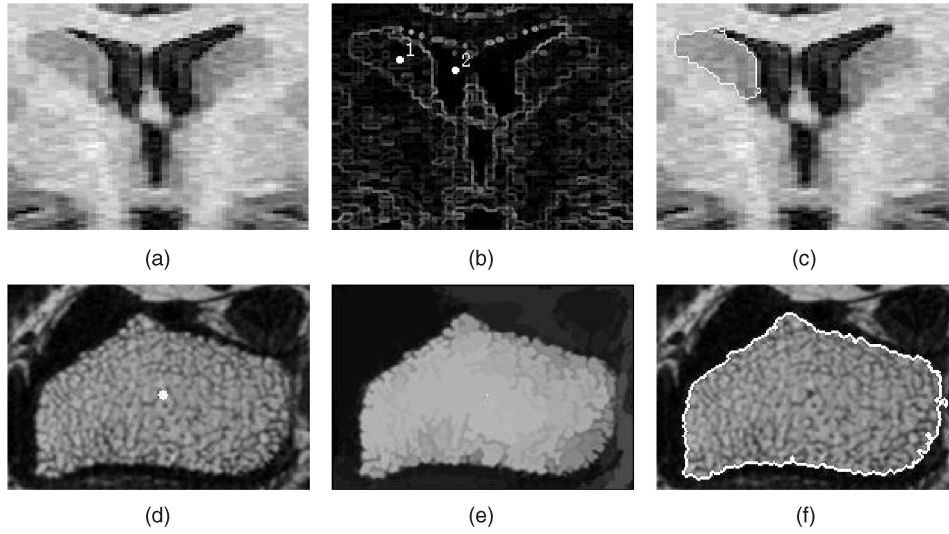


Fig. 5. Examples of segmentation using IFT-based watershed transform and morphological reconstruction. (a) A magnetic resonance (MR) image of a brain. (b) The gradient of image I' obtained from (a) by $I'(t) = 255 \exp[(I(t) - 140)^2 / (2 \times 30^2)]$, with a seed selected inside the left caudate nucleus (1), and another outside it (2). (c) The segmentation derived from the label map $L(t)$ of the IFT-watershed. (d) An MR image of a wrist with one seed pixel selected inside the bone. (e) The cost map $C(t)$ resulting from inferior morphological reconstruction of (d) with the given seed, followed by superior reconstruction with the image border pixels as seeds. (f) The bone is segmented by thresholding of (e).

one root r in each regional minimum. In this case, the full extent of each regional minimum is obtained by truncating the tree rooted at r to those pixels t with $I(t) = I(r)$.

6.2 The Watershed Transform and Morphological Reconstruction

Consider the image $\mathbf{I} = \{\mathcal{I}, I\}$ as a 3D surface, where $I(t)$ is the altitude of pixel t and the objects of interest are delimited by ridges which are higher than any other hills inside or outside the objects. This situation arises, for example, when $\mathbf{I}$ is a gradient-like image. In this case, we can separate the objects by choosing one or more seed pixels in each object (and background), with a distinct label for each object, and using the cost function f_{peak} , defined as the maximum pixel intensity along the path. This is a special case of f_{max} , where $h(t) = w(s, t) = I(t)$. The label map $L(t)$ will give the result of the segmentation.

This formulation captures the essential features of the *watershed transform* (WT) and several of its variants [20], [21], [6], [12]. There is yet no consensual definition for the WT [41]. It is informally described as a flooding process on the image surface, with a source of water at each seed; a barrier (*watershed line*) is erected wherever two bodies of water coming from distinct sources meet. However, the position of the watershed lines is not precisely defined in many situations, such as on plateaus and when two or more bodies of water overflow into an adjacent basin at the same time. In the IFT approach, this ambiguity is decided by the tie-breaking policy. The FIFO policy usually leads to a roughly fair partition of plateaus and flooded basins among competing sources (see Figs. 4a and 4c).

By restricting f_{peak} to a seed set $\mathcal{S} \subset \mathcal{I}$, the IFT computes a *watershed-from-markers* transform (WMT) (see Figs. 5a, 5b, and 5c). If we use arbitrary handicaps $h(t) > I(t)$ for all $t \in \mathcal{S}$, we get the WMT without marker imposition [21]—that is, some seeds may not become roots of the forest [12]. Marker imposition is achieved by assigning $h(t) < I(t)$ for all $t \in \mathcal{S}$ [6]; this solution is more efficient than the standard one based on homotopy change [21].

The *threshold decomposition* of an image $\mathbf{I}$ is the set $\mathcal{X}(\mathbf{I})$ of all connected components of pixel subsets of the form $\{t \in \mathcal{I} : I(t) \geq v\}$, for all $v \in \mathcal{V}$ [23], [12]. *Morphological reconstruction* operators are robust filters whose only effect is to eliminate or merge connected components from $\mathcal{X}(\mathbf{I})$. If we use f_{peak} with $h(t) > I(t)$ for all $t \in \mathcal{I}$, then $C(t)$ will be the *superior morphological reconstruction* of $\mathbf{I}$, which only merges components selected by the regional minima of the *marker function* $h(t)$; and $L(t)$ will be the catchment basins of the watershed transform of the reconstructed image $C(t)$ [12]. The *inferior reconstruction* is the dual operation, which only eliminates components from $\mathcal{X}(\mathbf{I})$. (See Figs. 5d, 5e, and 5f.)

The *local superior reconstruction* is a variant introduced by Falcão et al. [12] which fills up one or more basins, selected by a seed set $\mathcal{S}$, up to levels specified by given handicaps $h(t) > I(t)$ for $t \in \mathcal{S}$. The parameters $\mathcal{S}$ and $h(t)$ can be given by the user or selected automatically, as proposed by Vincent [22]. The required cost function f_{rec} is:

$$f_{\text{rec}}(\langle t \rangle) = h(t), \text{ if } t \in \mathcal{S}, \text{ and } +\infty \text{ otherwise,}$$

$$f_{\text{rec}}(\pi \cdot \langle s, t \rangle) = \begin{cases} f_{\text{rec}}(\pi), & \text{if } f_{\text{rec}}(\pi) > I(t), \\ +\infty, & \text{otherwise.} \end{cases} \quad (5)$$

Morphological reconstruction is the building block of many other image operations, such as *h-minima*, *h-maxima*, the *leveling operator*, *area opening* and *closing*, *alternate sequential filters by reconstruction*, *hole closing*, and *dome removal* [12], [22], [23], [24].

As in other implementations of the WT [20], [21], the queue structure can be simplified because, for f_{peak} and f_{rec} , the cost of a pixel never needs to be updated once it has been inserted in $\mathcal{Q}$ [6]. Note that some generalizations of watershed may not have this property.

6.3 Boundary Tracking

In the *boundary tracking* approach to image segmentation, the goal is to find an optimum curve that is constrained to pass through a given sequence $\langle \mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_k \rangle$ of k landmarks (pixel sets) on the object's boundary, in that order, starting in

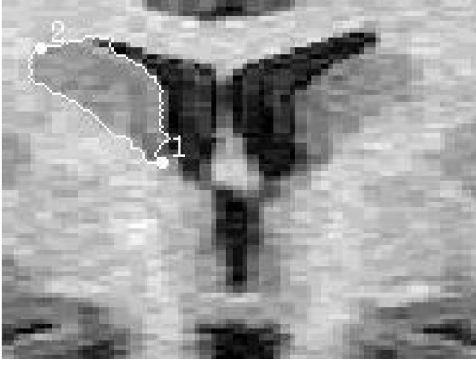


Fig. 6. Boundary tracking of the left caudate nucleus in an MR image of a brain, with single-pixel landmarks (1, 2, 1). The cost function is f_{sum} with the directional arc weights $w(s, t)$ described in Section 6.3, where $G(s, t)$ is the gradient of image I' used in Fig. 5.

T_1 and ending in T_k . (For instance, each set T_i could be a short stroke drawn by the user across the presumed object boundary.) In particular, if $T_1 = T_k$ is a single pixel, then the result will be an optimum *closed* path that satisfies those restrictions (see Fig. 6).

If the cost function f is MI, then the optimum path that satisfies those constraints consists of $k-1$ segments $\pi_1, \pi_2, \dots, \pi_{k-1}$, where each π_i is an f -optimum path connecting T_i to T_{i+1} . Therefore, we can solve this problem by $k-1$ executions of the IFT. Each execution i uses f^{T_i} as the cost function; except that, for $i > 1$, the handicap $h(t)$ is set to the cost $C_i(t)$, computed in the previous execution, if $t \in T_i$, or $+\infty$ otherwise. Each stage can be terminated as soon as the last pixel of the target set T_{i+1} is removed from the queue Q . The optimum path can be obtained from the predecessor maps $P_{k-1}, P_{k-2}, \dots, P_1$.

The function f should favor paths that go through boundary-like regions—for example, regions of high gradient, for objects with sharp high-contrast edges. The f_{sum} cost function is usually preferable to f_{max} because it favors shortest-distance jumps across regions where the boundary is not well defined.

We can also devise path costs that favor a specific orientation (clockwise or counterclockwise) for the boundary. Such a feature is useful to avoid nearby boundaries with similar contrast but opposite orientation [17]. For example, if the object is darker than the background, we can use $w(s, t) = K - \max\{G(s, t) \cdot \eta(s, t), 0\}$, where $G(s, t)$ is a gradient-like vector estimated at the midpoint of arc (s, t) ; $\eta(s, t)$ is the arc (s, t) rotated 90 degrees counterclockwise; and K is an upper bound for $|G(s, t) \cdot \eta(s, t)|$.

Boundary tracking is usually formulated in terms of heuristic graph search [32], [33], [46] or dynamic programming [31], [46]. The IFT approach covers both formulations and several interesting interactive extensions—such as *live-wire* [17], [35], [18], [19] and *live-lane* [17].

6.4 Euclidean Distance Transform and Related Operators

The *Euclidean distance transform* (EDT) of a given seed set S assigns to each image pixel t a value $C(t)$ which is the minimum Euclidean distance from t to S . (Actually, in order to avoid irrational numbers, the square of the distance is stored instead.)

The EDT is related to the concept of *discrete Voronoi diagram* (DVD), which is a partition of the image pixels into *discrete*

Voronoi regions. For any nonempty subset $\mathcal{X}$ of S , we define the discrete Voronoi region of $\mathcal{X}$ as the set $\mathcal{R}(\mathcal{X})$ of all image pixels t such that t is equidistant from all the seeds in $\mathcal{X}$ and is strictly closer to those seeds than to any other seed. We also define the *extended discrete Voronoi region* $\mathcal{R}^*(\mathcal{X}) = \bigcup\{\mathcal{R}(\mathcal{Y}) : \mathcal{Y} \subseteq \mathcal{X}\}$. In particular, for any seed $r \in S$, $\mathcal{R}(\{r\})$ consists of all image pixels whose centers lie in the interior of the exact (geometric) Voronoi polygon of S [34]; whereas $\mathcal{R}^*(\{r\})$ consists of the pixels inside or on the boundary of that polygon.

The obvious implementation of the EDT (or DVD) runs in time proportional to $n|S|$. This large cost motivated a search for faster propagation-style algorithms. Ragnemalm [39] proposed an algorithm for an approximate EDT, similar to Algorithm 2 with 8-connected adjacency and the Euclidean path-cost function $f_{\text{euc}}^S(\pi)$ defined in Section 4.3. When $|S| \geq 3$, this algorithm may not output the exact EDT, because the regions of the discrete Voronoi diagram may not be 8-connected, as observed by Danielsson [47]. (Indeed, the path-cost function f_{euc}^S is not smooth in this case.) On the other hand, Cuisenaire and Macq [40] observed that the exact EDT can be computed by Algorithm 2 if the adjacency radius is large enough. Lemma 6 captures this idea:

Lemma 6. *For any image diameter D , there is a radius $\rho \geq \sqrt{2}$ such that any pixel $t \notin S$ can be reached by a path $\pi = \tau \cdot \langle s, t \rangle$ that satisfies: (E1) $r = \text{org}(\pi)$ is one of the seeds closest to t in the Euclidean distance, (E2) all pixels along τ belong to $\mathcal{R}(\{r\})$, (E3) all arcs in π are directed strictly away from r , (E4) τ is 8-connected, and (E5) $d(s, t) \leq \rho$.*

The lemma is proven by taking $\rho \geq \rho^*(D)$, the maximum Hausdorff distance between $\mathcal{R}^*(\{r\})$ and the 8-connected component of $\mathcal{R}(\{r\})$ that contains r , for any $r \in S$. According to Cuisenaire and Macq [40], $\rho^*(D) \ll D$ (e.g., $\rho^* = 16$ for a 768×768 image).

Lemma 7. *If $\mathcal{A}$ is the Euclidean adjacency with radius $\rho \geq \rho^*(D)$, then f_{euc}^S is smooth.*

Proof. For each pixel t , if $t \in S$ then the path $\pi = \langle t \rangle$ is optimum for f_{euc}^S and satisfies the smoothness condition trivially. If $t \notin S$, let $\pi = \tau \cdot \langle s, t \rangle$ be a path from some seed r to t that satisfies the Conditions E1, E2, E3, E4, and E5 of Lemma 6. Because of Conditions E4 and E5, the path π is $\mathcal{A}$ -connected. Because of Condition E1, π is optimum for f_{euc}^S . Because of Condition E2, s lies in $\mathcal{R}(\{r\})$ and, therefore, τ also is optimum for f_{euc}^S , thus satisfying C2. Because of Condition E3, $d(r, s) \leq d(r, t)$ and, therefore, $f_{\text{euc}}^S(\tau) \leq f_{\text{euc}}^S(\pi)$, satisfying C1. Finally, let τ' be any optimum path leading to s ; since $s \in \mathcal{R}(\{r\})$, we must have $\text{org}(\tau') = r$ and, therefore, $f_{\text{euc}}^S(\tau' \cdot \langle s, t \rangle) = d(r, t) = f_{\text{euc}}^S(\pi)$, which is C3. $\square$

Cuisenaire and Macq also pointed out that one can correct Ragnemalm's approximate EDT by a postprocessing stage which considers all arcs whose Euclidean length is less than or equal to $\rho^*(D)$ and whose origin lies in a certain subset of the leaf pixels of the computed influence zones. Note, however, that propagation methods will hardly compete with other approaches based on the exact geometric Voronoi diagram, which can compute the exact EDT in linear time [48]. The algorithm by Cuisenaire and Macq will require $\Omega(n|\mathcal{A}|) = \Omega(n(\rho^*(\sqrt{n}))^2)$ time in the worst case, since the image diameter D is at least $\Omega(\sqrt{n})$, and it is possible that $\Omega(n)$ pixels will become seeds of the second stage. The asymptotic growth of

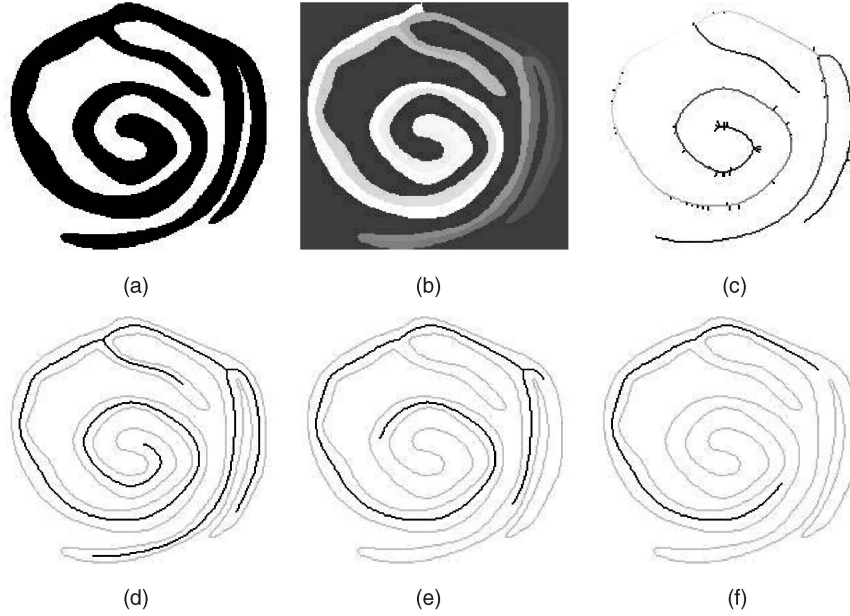


Fig. 7. (a) A binary image. (b) The label map $L(t)$ resulting from the quasi-IFT with f^S_{enc} , where the seeds are the boundary pixels with labels $\lambda(t)$ that increase sequentially along the contour. (c) A multiscale skeleton computed from (b) by local differencing. (d), (e), and (f) Skeletons computed by thresholding (c) at three different spatial scales.

$\rho^*(D)$ is not known, but from the table in [40] it seems to be at least $\Omega(\sqrt{D})$, which would imply total cost $\Omega(n^{3/2})$.

On the other hand, propagation algorithms seem mandatory in applications where it is more important to have 8-connected influence zones than exact distance values [14], [15], [16]. Examples include the computation of one-pixel-wide and connected multiscale skeletons (see Fig. 7) and the skeleton by influence zones (SKIZ) [14]. The former are used by Torres et al. to compute the salience points of a given contour [16].

In such applications, one could run Algorithm 2 with cost function f^S_{enc} and $\rho = \sqrt{2}$ (8-connected adjacency). Because of Lemma 2, the result will always be *some* partition of the image into 8-connected influence zones, even though the forest may not be optimum. However, because of Condition E4 of Lemma 6, it can be shown that the influence zone of each seed $r \in S$ will contain the subset of $\mathcal{R}(\{r\})$ which is 8-connected to r . So, in this sense, this “quasi-IFT” is an approximation of the EDT/DVD, which may be good enough for many applications.

Another alternative is to compute a *chamfer distance transform* (CDT), where the Euclidean distance is replaced by a piecewise-linear approximation. One can obtain an exact CDT by applying Algorithm 2 with the f_{sum} cost function and arc weights $w(s, t)$ that depend only on the vector $t - s$ [36]. For most applications, it suffices to use 8-connected adjacency with weights 5 for axial arcs and 7 for diagonal arcs [49]. One drawback of the CDT is that the resulting skeletons are more affected by rotations than the quasi-EDT above.

7 CONCLUSIONS AND CURRENT RESEARCH

We gave a precise definition of the image foresting transform, a proof of correctness of its basic algorithm (for a fairly general class of cost functions), and an efficient implementation (Algorithm 2). We pointed out the importance of the tie-breaking policy, especially for the watershed transform,

and introduced the LIFO variant (Algorithm 3), which led to an efficient way of locating regional minima. We also presented cost functions for watershed transforms, morphological reconstructions, boundary tracking, and EDT-related operators.

The IFT provides a common and well-founded framework for the design of image processing operators, which leads to correct, efficient, and flexible implementations. Algorithms 2 and 3 together with the examples of this paper are available through the Internet [50].

We are currently exploiting Algorithm 2 and variants to improve the efficiency of watershed transforms [25]. We are also developing novel IFT-based algorithms for automatic image segmentation [51], and paradigms for 3D segmentation of medical images at interactive speeds, such as the *differential IFT* [26], [52]. Finally, we are investigating parallel and hardware implementations of the IFT and its possible application to 3D multiscale skeletonization.

APPENDIX A

PROOFS OF THE LEMMAS

By induction on the length of π , Conditions C1, C2, and C3 can be extended to arbitrary prefixes of optimum paths. Namely, if f is a smooth path-cost function, then for any $t \in \mathcal{I}$ there is an optimum path π to t , such that, for any τ and δ with $\pi = \tau \cdot \delta$ we have

$$C1^*. f(\tau) \leq f(\pi),$$

$$C2^*. \tau \text{ is optimum,}$$

$$C3^*. \text{ for any optimum path } \tau' \text{ with same terminus as } \tau, \\ f(\tau' \cdot \delta) = f(\pi).$$

Lemma 3. Each execution of Step 2.2.1 of Algorithm 1 preserves Invariants H1, H2, and H3.

Proof. In this proof, for any variable or preposition V , we will denote by $\triangleleft V$ the value of V before a generic execution of Step 2.2.1 and by $\triangleright V$ its value after it. Thus, the effect of Step 2.2.1 is to change the predecessor map from $\triangleleft P$ into $\triangleright P$; and we must show that, as a consequence, $\triangleleft(H1 \wedge H2 \wedge H3)$ implies $\triangleright(H1 \wedge H2 \wedge H3)$.

By definition, Step 2.2.1 adds the arc (s, t) to $\mathcal{B}$ (i.e., $\triangleright \mathcal{B} = \triangleleft \mathcal{B} \cup \{(s, t)\}$). That step may change the predecessor map P for pixel t , but will have no effect on $P(r)$, for any pixel $r \neq t$. Since Step 2.2.1 has no effect on $\mathcal{Q}$ or $\mathcal{F}$, or on $P^*(s')$ for any $s' \in \mathcal{F}$, Invariants H1 and H2 are trivially preserved.

Now, observe that, for any r and at any point in the algorithm, $P(r)$ is either *nil* or a pixel of $\mathcal{F}$; so $P^*(r)$ is always in $\mathcal{F} \cup \{r\}$. Since t is in $\mathcal{Q}$, it follows that $\triangleright P^*(r) = \triangleleft P^*(r)$ for any $r \neq t$. Moreover, Step 2.2.1 only changes $P(t)$ if $f(\triangleright P^*(t)) < f(\triangleleft P^*(t))$. We conclude that $f(\triangleright P^*(t')) \leq f(\triangleleft P^*(t'))$, for any $t' \in \mathcal{Q}$.

Now, suppose that H3 is violated by Step 2.2.1, that is, there exists some pixel $t' \in \mathcal{Q}$ and some $(\triangleright \mathcal{B}, \triangleright P)$ -path π ending at t' with $f(\pi) < f(\triangleright P^*(t'))$. By the reasoning above, we should have also $f(\pi) < f(\triangleleft P^*(t'))$. On the other hand, given $\triangleleft H3$, the path π cannot be a $(\triangleleft \mathcal{B}, \triangleleft P)$ -path. However, since $\triangleright P^*(t') = \triangleleft P^*(t')$ for all $t' \neq t$, it follows that π must use the arc (s, t) , which is the only difference between $\triangleleft \mathcal{B}$ and $\triangleright \mathcal{B}$. Then, π must be the same path $P^*(s) \cdot \langle s, t \rangle$ considered in Step 2.2.1. However, that step guarantees that $f(\pi) \geq f(\triangleright P^*(t'))$, a contradiction. We conclude that Invariant H3 is preserved, too. $\square$

Lemma 4. If f is a smooth path-cost function, Step 2.1 preserves Invariants H1, H2, and H3.

Proof. In this proof, we will denote by $\triangleleft V$ the value of V before a generic execution of Step 2.1 and by $\triangleright V$ its value after it. Again, we must show that $\triangleleft(H1 \wedge H2 \wedge H3)$ implies $\triangleright(H1 \wedge H2 \wedge H3)$.

The effect of Step 2.1 is only to set $\triangleright \mathcal{F} = \triangleleft \mathcal{F} \cup \{s\}$, $\triangleright \mathcal{Q} = \triangleleft \mathcal{Q} \setminus \{s\}$. The step has no effect on the set $\mathcal{B}$, the forest P , or the paths P^* . Therefore, to show that Invariant H1 is preserved, we must show only that $P^*(s)$ is optimum.

Let π be an optimum path that ends at s . Since f is a smooth path-cost function, we can assume, without loss of generality, that π satisfies Conditions C1*, C2*, and C3*.

Let's consider first the case when $\text{org}(\pi) = r$ is in $\triangleleft \mathcal{Q}$. In that case, by the choice of s , we must have $f(P^*(s)) \leq f(P^*(r))$. Moreover, $\langle r \rangle$ is a $(\triangleleft \mathcal{B}, P)$ -path; by Invariant $\triangleleft(H3)$, it follows that $f(P^*(r)) \leq f(\langle r \rangle)$. On the other hand, by Condition C2*, the path $\langle r \rangle$ is optimum; by Condition C1*, we must have $f(\langle r \rangle) \leq f(\pi)$. It follows that $f(P^*(s)) \leq f(\pi)$, meaning that $P^*(s)$ is optimum.

Let's now consider the case where $\text{org}(\pi) \in \triangleleft \mathcal{F}$. Let τ be the longest prefix of π that is entirely contained in $\triangleleft \mathcal{F}$; let $\langle r', s' \rangle$ be the following arc, and δ the remainder of π , from s' to s . By Invariant H1, the path $\tau' = P^*(r')$ is optimum; by Condition C3*, the path $\tau' \cdot \langle r', s' \rangle \cdot \delta$ is also optimum. Also, by Condition C1*, we must have

$$f(\tau' \cdot \langle r', s' \rangle) \leq f(\pi). \quad (6)$$

Now, since $r' \in \triangleleft \mathcal{F}$ and $s' \in \triangleleft \mathcal{Q}$, the arc $\langle r', s' \rangle$ must have been added to $\triangleleft \mathcal{B}$ in some previous execution of Step 2.2 (right after r' was removed from $\mathcal{Q}$). It follows that $\tau' \cdot \langle r', s' \rangle$ is a $(\triangleleft \mathcal{B}, P)$ -path and by Invariant $\triangleleft(H3)$,

$$f(P^*(s')) \leq f(\tau' \cdot \langle r', s' \rangle). \quad (7)$$

Furthermore, by choice of s , we have $f(P^*(s)) \leq f(P^*(s'))$. Combining this fact with (6) and (7), we conclude that $f(P^*(s)) \leq f(\pi)$, meaning that $f(P^*(s))$ is indeed optimum. $\square$

ACKNOWLEDGMENTS

The authors would like to thank the MIPG, University of Pennsylvania, and the Department of Neurology, University of Campinas for the medical images used in the examples. This work was partially supported by CNPq (Procs. 302966/02-1 and 301016/92-5).

REFERENCES

- [1] E.F. Moore, "The Shortest Path through a Maze," *Proc. Int'l Symp. Theory of Switching*, pp. 285-292, Apr. 1959.
- [2] R. Bellman, "On a Routing Problem," *Quarterly of Applied Math.*, vol. 16, pp. 87-90, 1958.
- [3] E.W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269-271, 1959.
- [4] R.B. Dial, "Shortest-Path Forest with Topological Ordering," *Comm. ACM*, vol. 12, no. 11, pp. 632-633, Nov. 1969.
- [5] R.K. Ahuja, T.L. Magnanti, and J.B. Orlin, *Network Flows: Theory, Algorithms and Applications*. Prentice-Hall, 1993.
- [6] R.A. Lotufo and A.X. Falcão, "The Ordered Queue and the Optimality of the Watershed Approaches," *Math. Morphology and Its Applications to Image and Signal Processing*, vol. 18, pp. 341-350, June 2000.
- [7] R.A. Lotufo, A.X. Falcão, and F. Zampiroli, "IFT-Watershed from Gray-Scale Marker," *Proc. 15th Brazilian Symp. Computer Graphics and Image Processing*, pp. 146-152, Oct. 2002.
- [8] J.K. Udupa and S. Samarasekera, "Fuzzy Connectedness and Object Definition: Theory, Algorithms, and Applications in Image Segmentation," *Graphical Models and Image Processing*, vol. 58, pp. 246-261, 1996.
- [9] P.K. Saha and J.K. Udupa, "Relative Fuzzy Connectedness Among Multiple Objects: Theory, Algorithms, and Applications in Image Segmentation," *Computer Vision and Image Understanding*, vol. 82, pp. 42-56, 2001.
- [10] L.G. Nyúl, A.X. Falcão, and J.K. Udupa, "Fuzzy-Connected 3D Image Segmentation at Interactive Speeds," *Graphical Models*, vol. 64, no. 5, pp. 259-281, 2003.
- [11] B.S. Cunha, "Projeto de Operadores de Processamento e Análise de Imagens Baseados na Transformada Imagem-Floresta," Master's thesis, Universidade Estadual de Campinas, Instituto de Computação, Ago 2001.
- [12] A.X. Falcão, B.S. da Cunha, and R.A. Lotufo, "Design of Connected Operators Using the Image Foresting Transform," *Proc. SPIE Medical Imaging*, vol. 4322, pp. 468-479, Feb. 2001.
- [13] R.A. Lotufo, A.X. Falcão, and F.A. Zampiroli, "Fast Euclidean Distance Transform Using a Graph-Search Algorithm," *Proc. XIII Brazilian Symp. Computer Graphics and Image Processing*, pp. 269-275, Oct. 2000.
- [14] A.X. Falcão, L.F. Costa, and B.S. da Cunha, "Multiscale Skeletons by Image Foresting Transform and Its Applications to Neuromorphometry," *Pattern Recognition*, vol. 35, no. 7, pp. 1571-1582, Apr. 2002.
- [15] R.S. Torres, A.X. Falcão, and L.F. Costa, "Shape Description by Image Foresting Transform," *Proc. 14th Int'l Conf. Digital Signal Processing*, pp. 1089-1092, July 2002.
- [16] R.S. Torres, A.X. Falcão, and L.F. Costa, "A Graph-Based Approach for Multiscale Shape Analysis," Technical Report IC-03-03, Inst. of Computing, Univ. Campinas, Jan. 2003.
- [17] A.X. Falcão, J.K. Udupa, S. Samarasekera, S. Sharma, B.E. Hirsch, and R.A. Lotufo, "User-Steered Image Segmentation Paradigms: Live-Wire and Live-Lane," *Graphical Models and Image Processing*, vol. 60, no. 4, pp. 233-260, July 1998.
- [18] A.X. Falcão, J.K. Udupa, and F.K. Miyazawa, "An Ultra-Fast User-Steered Image Segmentation Paradigm: Live-Wire-on-the-Fly," *IEEE Trans. Medical Imaging*, vol. 19, no. 1, pp. 55-62, Jan. 2000.

- [19] A.X. Falcão and J.K. Udupa, "A 3D Generalization of User-Steered Live Wire Segmentation," *Medical Imaging Analysis*, vol. 4, no. 4, pp. 389-402, Dec. 2000.
- [20] L. Vincent and P. Soille, "Watersheds in Digital Spaces: An Efficient Algorithm Based on Immersion Simulations," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 13, no. 6, pp. 583-598, June 1991.
- [21] S. Beucher and F. Meyer, "The Morphological Approach to Segmentation: The Watershed Transformation," *Mathematical Morphology in Image Processing*, chapter 12, pp. 433-481, Marcel Dekker, 1993.
- [22] L. Vincent, "Morphological Area Opening and Closings for Greyscale Images," *Proc. Shape in Picture '92-NATO Workshop*, Sept. 1992.
- [23] L. Vincent, "Morphological Grayscale Reconstruction in Image Analysis," *IEEE Trans. Image Processing*, vol. 2, no. 2, pp. 176-201, Apr. 1993.
- [24] F. Meyer, "The Levelings," *Proc. Fourth Int'l Symp. Math. Morphology*, pp. 190-207, 1998.
- [25] R.A. Lotufo, A.X. Falcão, and F. Zampiroli, "IFT-Watershed from Gray-Scale Marker," Technical Report IC-02-12, Inst. of Computing, Univ. of Campinas, Dec. 2002.
- [26] A.X. Falcão and F.P.G. Bergo, "The Iterative Image Foresting Transform and Its Application to User-Steered 3D Segmentation," *Proc. SPIE Medical Imaging*, vol. 5032, pp. 1464-1475, Feb. 2003.
- [27] N. Deo and C. Pang, "Shortest-Path Algorithms: Taxonomy and Annotation," *Networks*, vol. 14, pp. 275-323, 1984.
- [28] T. Cormen, C. Leiserson, and R. Rivest, *Introduction to Algorithms*. MIT, 1990.
- [29] M. Mohri, "Semiring Frameworks and Algorithms for Shortest-Distance Problems," *J. Automata, Languages, and Combinatorics*, vol. 7, no. 3, pp. 321-350, 2002.
- [30] A. Frieze, "Minimum Paths in Directed Graphs," *Operational Research Quarterly*, vol. 28, no. 2, i, pp. 339-346, 1977.
- [31] U. Montanari, "On the Optimal Detection of Curves in Noisy Pictures," *Comm. ACM*, vol. 14, no. 5, pp. 335-345, 1971.
- [32] A. Martelli, "Edge Detection Using Heuristic Search Methods," *Computer Graphics and Image Processing*, vol. 1, pp. 169-182, 1972.
- [33] A. Martelli, "An Application of Heuristic Search Methods to Edge and Contour Detection," *Comm. ACM*, vol. 19, no. 2, pp. 73-83, 1976.
- [34] F.P. Preparata and M.I. Shamos, *Computational Geometry: An Introduction*. Springer 1985.
- [35] E.N. Mortensen and W.A. Barrett, "Interactive Segmentation with Intelligent Scissors," *Graphical Models and Image Processing*, vol. 60, pp. 349-384, 1998.
- [36] B.J.H. Verwer, P.W. Verbeek, and S.T. Dekker, "An Efficient Uniform Cost Algorithm Applied to Distance Transforms," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 11, no. 4, pp. 425-429, Apr. 1989.
- [37] M.Y. Sharaiha and N. Christofides, "A Graph-Theoretic Approach to Distance Transformations," *Pattern Recognition Letters*, vol. 15, pp. 1035-1041, Oct. 1994.
- [38] F. Meyer, "Topographic Distance and Watershed Lines," *Signal Processing*, vol. 38, pp. 113-125, 1994.
- [39] I. Ragnemalm, "Neighborhoods for Distance Transformations Using Ordered Propagation," *CVGIP: Image Understanding*, vol. 56, no. 3, pp. 399-409, 1992.
- [40] O. Cuisenaire and B. Macq, "Fast Euclidean Distance Transformation by Propagation Using Multiple Neighborhoods," *Computer Vision and Image Understanding*, vol. 76, no. 1, pp. 163-172, Nov. 1999.
- [41] J.B.T.M. Roerdink and A. Meijster, "The Watershed Transform: Definitions, Algorithms and Parallelization Strategies," *Fundamenta Informaticae*, vol. 41, pp. 187-228, 2000.
- [42] J.A. Sethian, "Curvature and the Evolution of Fronts," *Comm. Math. Physics*, vol. 101, pp. 487-499, 1985.
- [43] J.A. Sethian, "A Fast Marching Level Set Method for Monotonically Advancing Fronts," *Proc. Nat'l Academy of Science*, vol. 93, no. 4, pp. 1591-1595, 1996.
- [44] L. Bischof and R. Adams, "Seeded Region Growing," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 16, no. 6, pp. 641-647, June 1994.
- [45] R. Bellman, *Dynamic Programming*. Princeton Univ., 1957.
- [46] M. Sonka, R. Boyle, and V. Hlavac, *Image Processing, Analysis, and Machine Vision*. ITP, 1999.
- [47] P.E. Danielsson, "Euclidean Distance Mapping," *Computer Graphics and Image Processing*, vol. 14, pp. 227-248, 1980.
- [48] C.R. Maurer Jr., R. Qi, and V. Raghavan, "A Linear Time Algorithm for Computing Exact Euclidean Distance Transforms of Binary Images in Arbitrary Dimensions," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 25, no. 2, pp. 265-270, Feb. 2003.
- [49] G. Borgefors, "Distance Transformations in Digital Images," *Computer Vision, Graphics, and Image Processing*, vol. 34, pp. 344-371, 1986.
- [50] A. Falcão, J. Stolfi, and R. Lotufo, "The IFT Project," <http://www.ic.unicamp.br/~afalcão/ift.html>, 2003.
- [51] G. Castellano, R.A. Lotufo, A.X. Falcão, and F. Cendes, "Characterization of the Human Cortex in MR Images through the Image Foresting Transform," *Proc. IEEE Int'l Conf. Image Processing (ICIP)*, Sept. 2003.
- [52] F.P.G. Bergo and A.X. Falcão, "Interactive 3D Segmentation of Brain MR-Images with the Image Foresting Transform," Technical Report IC-03-16, Inst. of Computing, Univ. of Campinas, July 2003.



Alexandre X. Falcão received the BSc degree in electrical engineering from the University of Pernambuco, PE, Brazil, in 1988. In 1993, he received the Msc degree in electrical engineering from the University of Campinas, SP, Brazil. He received the doctorate degree in electrical engineering from the University of Campinas in 1996. He has worked in computer graphics and image processing since 1991. From 1994 to 1996, he worked at the University of Pennsylvania on interactive image segmentation for his doctorate. In 1997, he developed video quality evaluation methods for Globo TV, RJ, Brazil. He has been a professor at the Institute of Computing, University of Campinas since 1998. His research interests include image segmentation and analysis, volume visualization, content-based image retrieval, mathematical morphology, digital TV, and medical imaging applications.



Jorge Stolfi received the BE degree in electrical engineering in 1973, the MSc degree in computer sciences in 1979 from the University of São Paulo, and the PhD degree in computer science from Stanford University in 1989. He is presently a full professor at the Institute of Computing, University of Campinas, SP, Brazil. He has also worked as research engineer at the Digital Systems Research Center from 1989 to 1992. His research interests include image processing, computer graphics, computational geometry, numerical analysis, natural language processing, and their applications.



Roberto de Alencar Lotufo received the Electronic Engineering Diploma from Instituto Tecnológico de Aeronáutica, Brazil, in 1978, the MSc degree from the University of Campinas, UNICAMP, Brazil, in 1981, and the PhD degree in Electrical Engineering from the University of Bristol, United Kingdom, in 1990. He is a professor in the Department of Computer Engineering and Industrial Automation at the University of Campinas (UNICAMP), Brazil, where he has worked for since 1981. His principal interests are in the areas of image processing and analysis, mathematical morphology, image segmentation, and medical imaging. He is one of the main architects of two morphological toolboxes: MMach for Khoros, and SDC Morphology Toolbox for MATLAB. Professor Lotufo has published more than 50 refereed conference and journal papers.

► For more information on this or any other computing topic, please visit our Digital Library at <http://computer.org/publications/dlib>.